IMT Atlantique

Projet Procom

Argus

Guide d'utilisation

SCHOBERT Néo
EL AKRABA Othmane
AYOUBI Houcine
TRAORE Idrissa

Argus – Documentation

Table des matières

I Prérequis **2**

II Configuration **2**

III Démarrage avec Docker Compose **6**

IV Utilisation **7**

V Diagnostics courants **8**

I Prérequis

Dépendances logicielles

- Docker et Docker Compose
- Un serveur Ollama accessible (local ou distant) hébergeant un modèle compatible (voir §II)

Clés API

Argus nécessite deux clés API externes :

Service	Obligatoire	Usage
OpenRouter	**Oui**	Résolution de bibliothèques, enrichissement CVE, juges de benchmarking
NVD (NIST)	Non	Augmente le rate limit CVE de 10 req/min à 120 req/min

- **OpenRouter** : `https://openrouter.ai` → créer un compte → API Keys → créer une clé
- **NVD** : `https://nvd.nist.gov/developers/request-an-api-key` → formulaire simple, clé reçue par e-mail

II Configuration

Fichier `backend/.env`

Créer (ou éditer) le fichier `backend/.env` à la racine du dossier `backend/` :

```
# -- Obligatoire ------------------------------------------------
# Cle OpenRouter : resolution de bibliotheques, enrichissement CVE,
# modeles juges (benchmarking)
OPENROUTER_API_KEY=sk-or-v1-...

# -- Optionnel --------------------------------------------------
# Cle NVD : augmente le rate limit de 10 req/min a 120 req/min
# Sans cle : delai de 6s entre requetes CVE
# Avec cle  : delai de 0.6s
NVD_API_KEY=

# -- Modele on-premise ------------------------------------------
# URL de votre serveur Ollama (compatible OpenAI /v1)
ON_PREMISE_BASE_URL=http://<votre-serveur-ollama>/v1

# Modele a utiliser (voir contraintes ci-dessous)
```

```
ON_PREMISE_MODEL=devstral:24b

# Cle d'authentification -- laisser vide si Ollama sans auth
ON_PREMISE_API_KEY=

# -- Debug -----------------------------------------------------
# 0 : mode normal (logs minimaux)
# 1 : affiche chaque prompt envoye au LLM ET sa reponse brute dans
   stdout
DEBUG_AI=0
```

Listing 1 – backend/.env – configuration complete

Choix du modèle on-premise

Tous les modèles ne sont pas utilisables. Le modèle on-premise doit disposer d'une fenêtre de contexte suffisante et de bonnes capacités de raisonnement sur le code. En dessous de 24B paramètres, les modèles manquent les chemins de taint et peinent à produire du JSON structuré.

Modèle	Qualité	Remarque
`devstral:24b`	**Recommandé**	Spécialisé code, meilleur alignement avec les juges
`qwen3:32b`	Bon	Solide en raisonnement général + code
`gpt-oss:20b`	Déconseillé	Variance élevée entre exécutions, sous-évalue systématiquement
Autres modèles 24B+	À benchmarker	Utiliser le workflow Test pour évaluer objectivement tout nouveau modèle

Table 1 – Modèles on-premise testés avec Argus

Tout modèle de taille suffisante (24B paramètres minimum) exposant une API compatible OpenAI sur `/v1` peut être utilisé.

La variable DEBUG_AI

DEBUG_AI=1 active une journalisation complète de tous les échanges avec les LLMs : chaque prompt envoyé et chaque réponse brute sont affichés dans la sortie standard (`stdout`) du backend.

C'est l'outil de diagnostic principal pour comprendre :

- Pourquoi un modèle produit un score inattendu
- Si le JSON retourné est correctement formé
- Quelles informations CVE ont été injectées dans le prompt
- Si le modèle on-premise reçoit bien les données (et non un timeout silencieux)

En production, laisser DEBUG_AI=0. Avec DEBUG_AI=1, les logs peuvent atteindre plusieurs milliers de lignes pour un projet de taille moyenne et exposent le contenu des prompts.

Fichier `frontend/.env`

```
# URL du backend FastAPI
NEXT_PUBLIC_BACKEND_ENDPOINT=http://localhost:8000
```

Listing 2 – frontend/.env

Modifier l'URL selon l'environnement de déploiement (Cloud Run, serveur distant...).

Référence complète des variables d'environnement

Les tableaux suivants récapitulent l'ensemble des variables d'environnement requises ou optionnelles pour faire fonctionner Argus, avec leur description et un exemple de valeur.

Backend (`backend/.env`) :

Variable	Obligatoire	Description	Exemple
OPENROUTER_API_KEY	**Oui**	Clé API OpenRouter — résolution de bibliothèques, enrichissement CVE, juges de benchmarking	`sk-or-v1-...`
NVD_API_KEY	Non	Clé API NVD (NIST) — augmente le rate limit de 10 à 120 req/min. Laisser vide si absent	`xxxxxxxx-xxxx-...`
ON_PREMISE_BASE_URL	**Oui**	URL de base du ser-	`http://192.168.1.10:11434/v1`

Frontend (`frontend/.env`) :

Variable	Obligatoire	Description	Exemple
`NEXT_PUBLIC_BACKEND_ENDPOINT`	**Oui**	URL du backend FastAPI accessible depuis le navigateur. Préfixée `NEXT_PUBLIC_` car exposée côté client. Adapter à l'adresse publique en production	`http://localhost:8000`

Table 3 – Variables d'environnement – frontend

III Démarrage avec Docker Compose

```
# 1. Renseigner les variables d'environnement dans backend/.env
#    (OPENROUTER_API_KEY et ON_PREMISE_BASE_URL au minimum)

# 2. Construire et demarrer
cd backend
```

```
docker compose up --build
```

Listing 3 – Deploiement avec Docker Compose

Le backend est accessible sur le port **8000**, le frontend sur le port **3000**.

Persistance des données : les résultats de benchmarking sont stockés dans `backend/data/test_results.db`. Ce dossier est monté comme volume Docker — ne pas le supprimer entre les redémarrages.

IV Utilisation

Workflow Analyse — analyser un projet C++

1. Ouvrir `http://localhost:3000`
2. Cliquer sur **Select Project Folder** et sélectionner le dossier du projet C++
3. Argus parse automatiquement le `CMakeLists.txt`, construit le graphe d'appels et interroge la NVD
4. Le graphe s'affiche : les nœuds sont colorés selon le risque (vert → rouge selon le score 0–10)
5. Cliquer sur **Generate Global AI Report** pour lancer l'analyse on-premise complète
6. Cliquer sur un nœud pour afficher ses CVEs et générer un rapport détaillé (**Analyze with AI**)

Rappel confidentialité : dans ce workflow, le code source n'est jamais envoyé au cloud. Seuls des noms de fonctions et des descriptions CVE peuvent transiter vers OpenRouter pour la résolution de bibliothèques et l'enrichissement contextuel.

Workflow Benchmarking — évaluer un modèle

1. Naviguer vers `/tests` via l'icône de navigation
2. Cliquer sur **+ New**
3. Sélectionner le projet, le **modèle testé** (on-premise) et les **modèles juges**
4. Cliquer sur **Launch Test** — le test s'exécute en arrière-plan

Le résultat affiche les scores globaux, les métriques qualité par nœud, et le rapport de consensus.

Note : dans ce workflow, le code source est transmis aux juges cloud pour qu'ils puissent évaluer l'analyse. Réserver ce mode à l'évaluation de modèles sur des projets non confidentiels.

V Diagnostics courants

Symptôme	Cause probable et solution
Timeout sur les requêtes LLM	ON_PREMISE_BASE_URL incorrect ou serveur Ollama inaccessible
Score = 0.0 sur tous les nœuds	Modèle non disponible sur le serveur Ollama (vérifier avec DEBUG_AI=1)
Aucune CVE trouvée	Bibliothèque non reconnue ou rate limit NVD atteint (ajouter NVD_API_KEY)
"[object Object]" dans le rapport	Modèle retourne du JSON malformé — passer à DEBUG_AI=1 pour inspecter la réponse brute
Modèle absent du sélecteur de tests	Le modèle n'est pas dans le catalogue OpenRouter — le modèle on-premise est injecté automatiquement au démarrage

Table 4 – Diagnostics courants